

Amazon ML Challenge 2026 — Master AI Prompt Playbook

High-Precision Engineering Prompts for Google Antigravity

Use these prompts with Google Antigravity to generate bug-free, production-grade ML code tailored for our central S3 storage and Colab compute workflow.

1. How to Use Antigravity for Cross-Conversation Context

When starting a new conversation in Antigravity or handing off tasks between team members, **always paste the Master Context Header** at the start of your message. This allows Antigravity to instantly link context, data schemas, and prior decisions from previous conversations without repeating explanations.

Master Context Header (Copy & Paste at the Start of New Conversations)

```
Project: Amazon ML Challenge 2026 – Multilingual Business Entity Resolution
Repository: ShubhamJain17r/Amazon-ML-Hackathon
Reference Master Conversation: conversation://afe57c16-937c-4ea0-9b50-31b154b41a0a
Architecture:
- Cloud Storage: AWS S3 central data lake (s3://<BUCKET>/raw, filtered, candidates, features, models, submissions)
- Compute: Google Colab / Local Python 3.10+
- Version Control: GitHub ("Save a copy in GitHub" Colab integration)
- Evaluation Metric: Macro F_0.5 score across all Source 1 entities (Precision has 2x weight over Recall; singlet
```

TIP: ****Recommended Antigravity Slash Commands:**** - `/grill-me``: Use when designing new feature combinations or blocking heuristics to evaluate edge cases before coding. - `/goal``: Use when asking Antigravity to build or refactor an entire multi-step pipeline end-to-end. - `/learn``: Use after finding a high-scoring threshold or optimal hyperparameter set to persist that knowledge for future sessions.

2. Stage-by-Stage Antigravity Prompts

Stage 0: Colab Environment & Central S3 Connectivity Verification

Target: Colab Notebook (`notebooks/<your_name>/00_s3_setup_test.ipynb`)

Antigravity Prompt:

[Master Context: conversation://afe57c16-937c-4ea0-9b50-31b154b41a0a]

Act as a Principal Cloud ML Engineer.

Write a comprehensive, self-contained Python validation script to run as the first cell in a Google Colab notebook.

Requirements:

1. Environment Check:

- Check if 'pandas', 'numpy', 'rapidfuzz', 'lightgbm', 'pyarrow', 'boto3', 's3fs', 'duckdb' are installed. If

2. Credentials & Authentication:

- Read AWS credentials securely from Google Colab userdata secrets ('AWS_ACCESS_KEY_ID', 'AWS_SECRET_ACCESS_KEY').
- Provide clear instructions in an exception block if secrets are missing.

3. S3 Read/Write Integrity Audit:

- Initialize s3fs.S3FileSystem().
- Create a small diagnostic DataFrame with columns: ['test_id', 'timestamp', 'status'].
- Write it to 's3://<BUCKET>/diagnostics/ping.parquet' using pyarrow.
- Read it back from S3, verify row count matches, and print success latency in milliseconds.
- Delete the temporary test parquet to leave no clutter.

4. Hardware & Resource Audit:

- Print total available system RAM (in GB), CPU core count, and GPU availability (e.g. NVIDIA T4 or CPU).
- Display a formatted PASS/FAIL summary table.

Stage 1: Exploratory Data Analysis & Multilingual Text Audit

Target: Colab Notebook ([notebooks/<your_name>/01_eda_multilingual.ipynb](#))

Antigravity Prompt:

[Master Context: conversation://afe57c16-937c-4ea0-9b50-31b154b41a0a]

Act as a Senior Data Scientist specializing in Multilingual NLP and Entity Resolution.

Write a clean, memory-conscious Google Colab script to perform exploratory data analysis directly on the training

Data Specifications:

- Source 1: ~2.2M records; Source 2 & 3: ~10.3M records combined; Ground Truth: ~1.8M mappings.
- Columns: 'entity_id', 'name', 'address', 'city', 'state', 'postal_code', 'country'.
- TSVs are tab-separated (sep='\\t').

Requirements:

1. Stream a 50,000-row sample of 'train_source1.tsv' and 'train_ground_truth.tsv' using pandas and s3fs.
2. Statistical Profile:
 - Country breakdown (frequency & percentage).
 - Null and whitespace-only value rates per column.
 - Singleton rate in ground truth (entities where 'matched_entity_ids' is empty/NaN).
3. Multilingual Character Detection:
 - Detect and report frequency of Hindi Devanagari Unicode characters (\\u0900-\\u097F).
 - Detect and report European accented characters (for French test set readiness).
 - Provide 5 real examples of noisy entity names (e.g., Devanagari script, punctuation spam, company abbreviations).
4. Address Analysis:
 - Character length distribution of address strings.
 - Token count distribution.
5. Actionable Recommendations:
 - Print a bulleted summary of specific cleaning rules required based on the observed data.

Stage 2: Multilingual Normalization & Suffix Filtering Module

Target: Python Module / Colab Cell (`src/normalization.py`)

Antigravity Prompt:

```
[Master Context: conversation://afe57c16-937c-4ea0-9b50-31b154b41a0a]
```

Act as a Senior NLP Software Engineer.

Write an ultra-fast, robust string normalization module ('src/normalization.py') tailored for the Amazon dataset below:

Dataset Realities:

- The data contains noisy English, Indian English, Hindi Devanagari text, and French accented text in thousands of records
- Legal suffixes vary widely (e.g., "Pvt. Ltd.", "Private Limited", "Inc.", "Corp.", "LLC", "LLP", "S.A")
- Address abbreviations vary (e.g., "Rd.", "Road", "St", "Street", "Ave", "Avenue", "Blvd").

Requirements:

1. 'clean_business_name(text: str) -> str':
 - Return empty string if input is None, NaN, or non-string.
 - Perform Unicode NFKD normalization to cleanly separate accents.
 - Convert to lowercase and strip leading/trailing whitespace.
 - Standardize business designations: map ('private limited', 'pvt ltd', 'pvt. ltd.', 'p ltd') -> 'private limited'
 - Retain standard alphanumeric characters, whitespace, and Hindi Devanagari range (\u0900-\u097F).
 - Collapse multiple consecutive whitespace characters into a single space.
2. 'clean_address(text: str) -> str':
 - Standardize street types: ('road', 'rd', 'rd.') -> 'road'; ('street', 'st', 'st.') -> 'street'; ('highway', 'hwy', 'hwy.') -> 'highway'
 - Normalize postal codes and remove excessive punctuation.
3. Unit Tests:
 - Include 6 automated test assertions at the bottom covering: None/empty inputs, Hindi names, French accents, etc.
 - Ensure the module executes in < 0.05ms per record.

Stage 3: Country-Partitioned Inverted Index Blocking Pipeline

Target: Python Module (`src/blocking.py`)

Antigravity Prompt:

[Master Context: conversation://afe57c16-937c-4ea0-9b50-31b154b41a0a]

Act as an Expert Search & Entity Resolution Architect.

We need to generate candidate matching pairs between Source 1 (~2.2M rows) and Source 2+3 (~10.3M rows) for the A
Pairwise Cartesian product is $O(N*M)$ (~2.2e13 comparisons) and will crash. We need candidate blocking that retrie

Requirements:

1. Country Partitioning:

- Partition records strictly by 'country' (US, IN, FR). Never cross-compare different countries!

2. Inverted Index Blocking:

- Within each country, build an inverted index on Source 2+3 normalized name tokens.
- Filter out stopwords and short tokens (< 3 characters).
- For each Source 1 record, query the inverted index using its tokens.
- Rank candidates by shared token count and select top 'max_candidates' (default 5).

3. Memory Safety:

- Process in streaming batches (e.g., 50,000 Source 1 records at a time) to prevent RAM spikes on Colab.
- Explicitly call 'gc.collect()' between batches.

4. Output Schema:

- Return DataFrame with columns: ['source1_entity_id', 'candidate_entity_id', 's1_name', 's23_name', 's1_addr']

5. Direct S3 Parquet Export:

- Save candidate pairs to 's3://<BUCKET>/candidates/train_candidates_sample50k.parquet' (or full).

Stage 4: RapidFuzz C++ Similarity Feature Extraction Engine

Target: Python Module (`src/features.py`)

Antigravity Prompt:

[Master Context: conversation://afe57c16-937c-4ea0-9b50-31b154b41a0a]

Act as a High-Performance ML Engineer.

Write an optimized feature extraction pipeline using the C++ RapidFuzz engine to compute similarity metrics on ca

Input Data:

- Candidate pairs DataFrame from S3 candidates folder.
- Ground truth set: 's3://<BUCKET>/raw/train_ground_truth.tsv'.

Requirements:

1. Ground Truth Lookup:
 - Load ground truth into a Python set of (s1_id, candidate_id) tuples for O(1) membership check.
2. Vectorized RapidFuzz Computations:
 - Pre-allocate numpy arrays with downcasted dtypes (float32, int16, int8) to cut RAM consumption by 50%.
 - Compute:
 - * 'name_ratio': fuzz.ratio / 100.0 (float32)
 - * 'name_partial': fuzz.partial_ratio / 100.0 (float32)
 - * 'name_token_sort': fuzz.token_sort_ratio / 100.0 (float32)
 - * 'name_token_set': fuzz.token_set_ratio / 100.0 (float32)
 - * 'addr_ratio': fuzz.ratio / 100.0 (float32)
 - * 'addr_partial': fuzz.partial_ratio / 100.0 (float32)
 - * 'addr_token_set': fuzz.token_set_ratio / 100.0 (float32)
 - * 'name_len_diff': abs(len(s1) - len(s23)) (int16)
 - * 'addr_len_diff': abs(len(s1) - len(s23)) (int16)
3. Leak-Proof GroupShuffleSplit:
 - Split into 80% train and 20% validation, strictly grouped by 'source1_entity_id'.
 - Validate with an assertion that len(set(train_s1).intersection(set(val_s1))) == 0.
4. Export:
 - Write directly to 's3://<BUCKET>/features/sample_50k/train_candidates.parquet' and 'val_candidates.parquet'

Stage 5: LightGBM Model Training with Class Imbalance Weighting

Target: Colab Notebook ([notebooks/<your_name>/02_train_lightgbm.ipynb](#))

Antigravity Prompt:

[Master Context: conversation://afe57c16-937c-4ea0-9b50-31b154b41a0a]

Act as a Kaggle Grandmaster & Gradient Boosting Specialist.

Write a production training notebook cell that loads feature Parquets directly from AWS S3 and trains a LightGBM

Challenge Conditions:

- True matches (label=1) are heavily imbalanced (~5-8% positive rate).
- Evaluation metric is Macro F_{0.5} (Precision has 2x weight over Recall).

Requirements:

1. Load 'train_candidates.parquet' and 'val_candidates.parquet' from 's3://<BUCKET>/features/full/' (or sample_50)
2. Configure LightGBM:
 - 'objective': 'binary'
 - 'metric': 'binary_logloss'
 - 'scale_pos_weight': 3.5 (compensates for class imbalance)
 - 'learning_rate': 0.05
 - 'num_leaves': 31
 - 'subsample': 0.8
 - 'colsample_bytree': 0.8
 - 'n_estimators': 800
3. Training & Callbacks:
 - Early stopping on validation logloss (stopping_rounds=50).
 - Display evaluation metrics every 50 iterations.
4. Feature Importance:
 - Print top features ranked by gain and split count.
5. S3 Model Checkpointing:
 - Serialize model using joblib and upload to 's3://<BUCKET>/models/lgbm_model_latest.pkl'.

Stage 6: Precision-Heavy Macro F_{0.5} Threshold Tuning & Singleton Logic

Target: Colab Notebook ([notebooks/<your_name>/03_threshold_tuning.ipynb](#))

Antigravity Prompt:

[Master Context: conversation://afe57c16-937c-4ea0-9b50-31b154b41a0a]

Act as an Expert Competition Evaluator.

The Amazon ML Challenge 2026 evaluates using per-entity Macro F_{0.5} score:

$$F_{0.5} = (1.25 * \text{Precision} * \text{Recall}) / (0.25 * \text{Precision} + \text{Recall})$$

Precision is weighted 2x more than recall! False positive matches severely hurt the score.

Furthermore, singletons (entities with no true matches) receive a score of 1.0 if predicted empty (""), but 0.0 if

Requirements:

1. Write a standalone Python function 'evaluate_macro_f05(ground_truth_map, predictions_map, all_s1_ids)' that co
2. Sweep probability thresholds from 0.40 to 0.94 in steps of 0.02.
3. For each threshold, compute:
 - Overall Precision, Recall, and Macro F_{0.5}.
 - Singleton accuracy rate (percentage of singletons correctly predicted as empty).
4. Identify the winning threshold and print a formatted comparative table.
5. Provide commentary on why the optimal threshold is significantly higher than 0.50.

Stage 7: Test Inference, Output Formatting & Submission Validation

Target: Colab Notebook ([notebooks/<your_name>/04_submission.ipynb](#))

Antigravity Prompt:

```
[Master Context: conversation://afe57c16-937c-4ea0-9b50-31b154b41a0a]
```

Act as an MLOps Competition Engineer.

Generate the official competition TSV submission files for the Amazon ML Challenge 2026 from test predictions.

Strict Competition Specifications:

- Two TSV files required:
 - 'output/matching_results.tsv' with columns: ['source1_entity_id', 'matched_entity_ids']
 - 'output/candidate_pairs.tsv' with columns: ['source1_entity_id', 'candidate_entity_ids']
- Exactly 1,732,544 rows in both files (every entity in 'test_source1.tsv' must appear exactly once).
- If an entity has multiple matches/candidates, join them with commas (e.g., "id1,id2").
- If an entity has no match above the optimal threshold, provide an empty string ("").
- Final matched_entity_ids MUST be a strict subset of candidate_entity_ids for every row.
- Run the local validator: 'python3 utils/validate_submission.py --matching output/matching_results.tsv --candidate output/candidate_pairs.tsv'
- Upload copies of both validated TSV files to 's3://<BUCKET>/submissions/'.

Stage 8: Error Recovery & Debugging Prompt

Target: Anytime code crashes or raises an exception

Antigravity Prompt:

```
[Master Context: conversation://afe57c16-937c-4ea0-9b50-31b154b41a0a]
```

I encountered an error during execution on Google Colab.

Here is the exact error traceback:

```
[PASTE TRACEBACK HERE]
```

Here is the code block that caused it:

```
[PASTE CODE BLOCK HERE]
```

Current Execution Context:

- Environment: Google Colab (Python 3.10)
- AWS Storage: S3 central bucket via s3fs / pyarrow
- Data Batch / Shape: [DESCRIBE DATA INPUT]

Please:

- Identify the root cause in 1-2 clear sentences.
- Provide the complete, drop-in replacement code block.
- Confirm there are no hidden memory leaks or index misalignment bugs.